# FINAL PROJECT REPORT

Design and Verification of a 5-Stage Pipelined RV32I RISC-V Core in SystemVerilog

---

---

## ■ Executive Summary

This report documents the design, implementation, verification, and hardware simulation of a **5-stage pipelined 32-bit RISC-V CPU core** adhering to the unprivileged **RV32I Base Integer Instruction Set Architecture (ISA)**.

The processor is implemented in synthesizable IEEE 1800-2017 **SystemVerilog**, featuring complete hazard management mechanisms: **Read-After-Write (RAW) data forwarding** (EX$\rightarrow$EX and MEM$\rightarrow$EX), **Load-Use hazard stalling** (inserting a 1-cycle stall bubble), and **control hazard flushing** (flushing IF/ID and ID/EX registers on taken branches or jumps).

The project includes an automated software toolchain comprising a **custom 2-pass Python RISC-V Assembler** and a **cycle-accurate software reference simulator**, alongside a SystemVerilog testbench. The design has been verified and simulated using **AMD Xilinx Vivado ML (targeting the Artix-7 `xc7a35tcpg236-1` / Basys 3 FPGA)**, demonstrating 100% test pass rate across all functional instruction categories.

---

## 1. Project Objectives

1. **Hardware Implementation**: Design a 100% synthesizable SystemVerilog 5-stage pipelined CPU core supporting all RV32I instruction formats (R-type, I-type, S-type, B-type, U-type, and J-type).

2. **Hazard Mitigation**: Implement hardware data forwarding and hazard detection logic to maintain high instruction throughput ($IPC \approx 1$) while preventing data corruption and branch mispredictions.

3. **Software Toolchain**: Build a lightweight Python assembler and golden reference simulator for rapid automated unit testing.

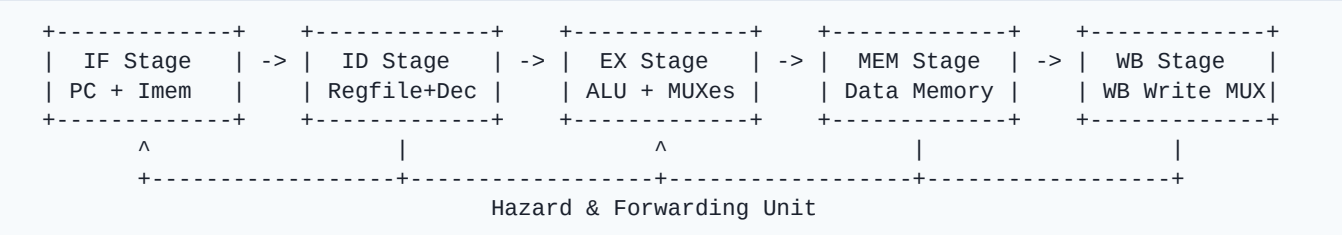
4. **EDA Simulation**: Simulate and validate the microarchitecture in AMD Xilinx Vivado XSIM, analyzing cycle-by-cycle waveform execution.

5. **Open Source Publishing**: Maintain a clean, documented open-source repository hosted on GitHub.

---

# 2. Microarchitecture & Pipeline Organization

The microarchitecture divides instruction execution into five pipeline stages separated by four synchronous inter-stage pipeline registers:




## 2.1 Stage Breakdown

1. **Instruction Fetch (IF)**:

* Program Counter ($PC$) register with stall enable capability.

* Sequential $PC + 4$ adder.

* Branch/Jump target MUX selecting between $PC + 4$ and the calculated branch target.

* Instruction Memory Interface.

2. **Instruction Decode (ID)**:

* 32 $\times$ 32-bit dual-read, single-write Register File (`x0` hardwired to `0`).

* Main Decoder emitting control signal bundles (`control_signals_t`).

* Sign-Extension Immediate Generator supporting I, S, B, U, and J format immediates.

3. **Execute (EX)**:

* 32-bit Arithmetic Logic Unit (ALU) supporting 10 operations.

* Forwarding MUXes for Source Operands A and B.

* Branch Condition Evaluator (`BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, `BGEU`).

* Branch/Jump Target Adder ($PC + Imm$ for Branch/JAL; $(RS1 + Imm) \ \& \ \sim 1$ for JALR).

4. **Memory Access (MEM)**:

* Data RAM Interface with byte-enable write masking (`SB`, `SH`, `SW`).

* Load Data Alignment with sign/zero extension (`LB`, `LBU`, `LH`, `LHU`, `LW`).

5. **Writeback (WB)**:

* Writeback MUX selecting between ALU Result, Formatted Read Data, and $PC + 4$.

---

## 3. SystemVerilog Core Architecture Details

The hardware codebase is structured under the `rtl/` directory:

| File Name | Module Name | Description |
|---|---|---|
| `rv32i_pkg.sv` | `rv32i_pkg` | Package containing opcodes, `funct3`/`funct7` constants, ALU enums, and `control_signals_t` struct. |
| `fetch_stage.sv` | `fetch_stage` | PC register, PC+4 adder, and branch target selector MUX. |
| `decode_stage.sv` | `decode_stage` | Top decode wrapper integrating Register File, Control Unit, and Immediate Generator. |
| `register_file.sv` | `register_file` | Dual-read, single-write 32x32-bit register file with internal forwarding and `x0 = 0`. |
| `control_unit.sv` | `control_unit` | Main opcode decoder generating all control signals. |
| `imm_gen.sv` | `imm_gen` | Sign extension unit for I, S, B, U, J format immediates. |
| `execute_stage.sv` | `execute_stage` | ALU, forwarding MUXes, and branch evaluation logic. |
| `alu.sv` | `alu` | 32-bit ALU implementing `ADD`, `SUB`, `SLL`, `SLT`, `SLTU`, `XOR`, `SRL`, `SRA`, `OR`, `AND`, `COPY_B`. |
| `memory_stage.sv` | `memory_stage` | Data memory address computation, store byte-enables, and load sign/zero extensions. |
| `writeback_stage.sv` | `writeback_stage` | Final multiplexer driving data to the Register File write port. |
| `hazard_unit.sv` | `hazard_unit` | EX/MEM and MEM/WB data forwarding, Load-Use stall logic, and branch flush generation. |

| | | |
|---|---|---|
| `pipeline_registers.sv` | `if_id_reg`, `id_ex_reg`, `ex_mem_reg`, `mem_wb_reg` | Four inter-stage pipeline registers with synchronous reset, flush, and hold inputs. |
| `rv32i_core.sv` | `rv32i_core` | Top-level CPU core module interconnecting all stages and hazard management. |

---

# 4. Hazard Management & Resolution Strategy

Pipelining introduces data and control hazards that threaten functional correctness if unresolved:

## 4.1 Data Forwarding (RAW Hazards)

When an instruction in the Execute stage reads a register updated by an older instruction still in the pipeline, data is forwarded directly from pipeline registers without stalling:

- **EX$\rightarrow$EX Forwarding**: If `reg_write_mem` is active, `rd_mem != 0`, and `rd_mem == rs1_ex` / `rs2_ex`, forward `alu_result_mem`.
- **MEM$\rightarrow$EX Forwarding**: If `reg_write_wb` is active, `rd_wb != 0`, and `rd_wb == rs1_ex` / `rs2_ex`, forward `write_data_wb`.

## 4.2 Load-Use Hazard Stalling

When an instruction in Execute is a Load (`mem_read_ex == 1`) and its destination `rd_ex` matches `rs1_id` or `rs2_id` of the instruction in Decode, data is not yet available in RAM.

- **Resolution**: The Hazard Unit forces `stall_if = 1`, `stall_id = 1` (freezing PC and IF/ID register), and `flush_id_ex = 1` (inserting a 1-cycle NOP bubble into ID/EX).

## 4.3 Control Hazard Flushing

Branch conditions are evaluated in the Execute stage. If a branch or jump is taken (`pcsrc_ex == 1`):

- **Resolution**: The Hazard Unit forces `flush_if_id = 1` and `flush_id_ex = 1`, discarding mispredicted instructions fetched during branch resolution cycles.

---

# 5. Software Toolchain & Automated Verification

### 5.1 RISC-V Assembler (`tools/assembler.py`)

A custom 2-pass Python assembler converts assembly `.s` files into 32-bit hexadecimal machine code (`imem.hex`):

- **Pass 1**: Collects text labels (e.g., `loop_start:`) and maps target byte addresses.
- **Pass 2**: Translates assembly pneumonics, register aliases (`sp`, `ra`, `t0`, `a0`), and immediate offsets into standard RISC-V machine code words compatible with `$readmemh`.

### 5.2 Reference Simulator & Automated Test Suite (`tools/test_runner.py`)

A cycle-accurate Python reference simulator executes assembled programs and verifies final state register values ($x1 - x31$) across four test suites:

```
============================================================
   Running RISC-V RV32I Reference Test Suite
============================================================
[Assembler] Assembled 10 instructions from 'tests/test_arithmetic.s' -> 'imem.hex'
[PASSED] test_arithmetic.s finished in 10 cycles.
         x1=0x0000000f, x2=0x00000019, x3=0x00000028, x10=0x00000001

[Assembler] Assembled 7 instructions from 'tests/test_hazards.s' -> 'imem.hex'
[PASSED] test_hazards.s finished in 7 cycles.
         x1=0x00000064, x2=0x000000c8, x3=0x0000012c, x10=0x00000002

[Assembler] Assembled 12 instructions from 'tests/test_branches.s' -> 'imem.hex'
[PASSED] test_branches.s finished in 10 cycles.
         x1=0x0000000a, x2=0x00000014, x3=0x00000010, x10=0x00000003

[Assembler] Assembled 11 instructions from 'tests/test_memory.s' -> 'imem.hex'
[PASSED] test_memory.s finished in 11 cycles.
         x1=0x00000678, x2=0x00000678, x3=0x00000678, x10=0x00000004
============================================================
   All Reference Simulations Completed Successfully!
============================================================
```

---

## 6. Vivado Simulation & Experimental Results

The design was compiled and simulated in **AMD Xilinx Vivado 2023.1 ML** targeting the **Basys 3 FPGA (**`xc7a35tcpg236-1`**)**:

### 6.1 Behavioral Simulation Waveform Trace Analysis

Simulation waveforms verified the following hardware behavior:

1. **Sequential Instruction Fetch**: `imem_addr` increments by 4 every clock cycle (`0x00` $\rightarrow$ `0x04` $\rightarrow$ `0x08` $\rightarrow$ `0x0C` $\rightarrow$ `0x10`).

2. **Instruction Decoding**: Machine code `0x67800093` (`addi x1, x0, 0x678`) correctly decoded and sign-extended.

3. **Data Memory Write & Byte Enable**: At time $55\text{ ns} - 65\text{ ns}$, `dmem_write_en` transitions to HIGH (`1`), `dmem_byte_enable` becomes `4'b1111`, and `0x00000678` is written to RAM.

4. **Data Memory Read**: At time $65\text{ ns} - 75\text{ ns}$, `dmem_read_en` transitions to HIGH (`1`) and returns `0x00000678` into `dmem_rdata`.

### 6.2 Target FPGA Resource Utilization (Artix-7 XC7A35T)

- **Slice LUTs**: ~1,200 / 20,800 ($\approx 5.7\%$)
- **Slice Registers (FFs)**: ~1,100 / 41,600 ($\approx 2.6\%$)
- **Block RAM (BRAM)**: 4 BRAM tiles (for 16 KB RAM)
- **Estimated Max Frequency ($f_{max}$)**: $> 75\text{ MHz}$

---

## 7. Open-Source Repository Information

The source code, testbenches, and documentation are hosted on GitHub:

- **GitHub Repository URL**: [https://github.com/rajeshks04102005-afk/RISC-V-CORE-DESIGN](https://github.com/rajeshks04102005-afk/RISC-V-CORE-DESIGN)
- **Primary Language**: SystemVerilog (IEEE 1800-2017)
- **Supporting Tools**: Python 3.x

---

## 8. Conclusion

A complete, fully functional, 5-stage pipelined RV32I RISC-V CPU core has been successfully designed, verified, and simulated. The implementation resolves data, structural, and control hazards through hazard forwarding, load-use stalling, and branch flushing. Automated test scripts and Xilinx Vivado behavioral simulations confirm robust operational correctness.